

OpenClaw Peripheral Hardware Configuration

OpenClaw Peripheral Hardware Configuration

1. Course Overview
2. Hardware Configuration File Description
 - 2.1 Configuration File Location
 - 2.2 Current Configuration File Coverage
 - 2.3 Peripheral Wiring Diagram
3. DHT Hardware Test
 - 3.1 Verification Method
4. RGB Hardware Test
 - 4.1 Verification Method
5. Servo Hardware Test
 - 5.1 Verification Method
6. OpenClaw Firmware Serial Port Settings
 - 6.1 Current Default Values
 - 6.2 Modification Example
 - 6.3 Verification Method
7. OLED Hardware Test
 - 7.1 Current Default Values
 - 7.2 Scenarios Suitable for Modification
 - 7.3 Common Modification Items
 - Modify Resolution
 - Modify I2C Address
 - Modify Rotation Direction
 - 7.4 Verification Method
8. Camera Hardware Test
 - 8.1 Configuration
 - 8.2 Check Port
 - 8.3 Photo Verification Method
 - 8.4 Adjustable Parameter Description
9. Development Environment Description:
 - 9.1 Download VScode
 - 9.2 Install Corresponding Plugins, Note as Pre-Release Version
 - 9.3 Enter Docker Container

1. Course Overview

This course will introduce OpenClaw's hardware-related default configuration files before use, focusing on how to modify the underlying default parameters according to the actual situation of your peripherals, and complete basic verification.

- Use underlying commands to verify if modifications take effect
- Must enter docker container before use!!!

2. Hardware Configuration File Description

2.1 Configuration File Location

OpenClaw's current hardware default configuration file is located at this path in the docker container:

```
/root/openclaw_port/hardware_ha1_demo/hardware_config.py
```

The default uses the factory preset image. If you are only following the accompanying tutorial and using the factory-provided hardware, modifications are usually not needed.

2.2 Current Configuration File Coverage

`hardware_config.py` currently contains three main types of default configurations:

- OpenClaw firmware serial port default parameters
- OLED default parameters

These default values are read by underlying command-line tools, such as:

- `rgb`
- `oled`
- `dht`

Important notes:

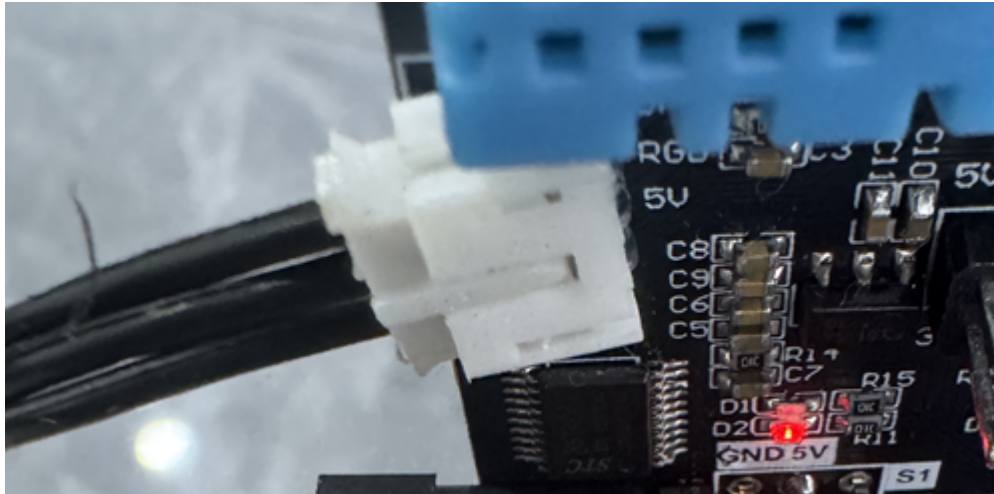
- **DHT and RGB are driven directly by OpenClaw expansion board firmware**
- **OLED is driven by the mainboard's I2C pins**
- **CSI camera requires checking hardware connection**

2.3 Peripheral Wiring Diagram

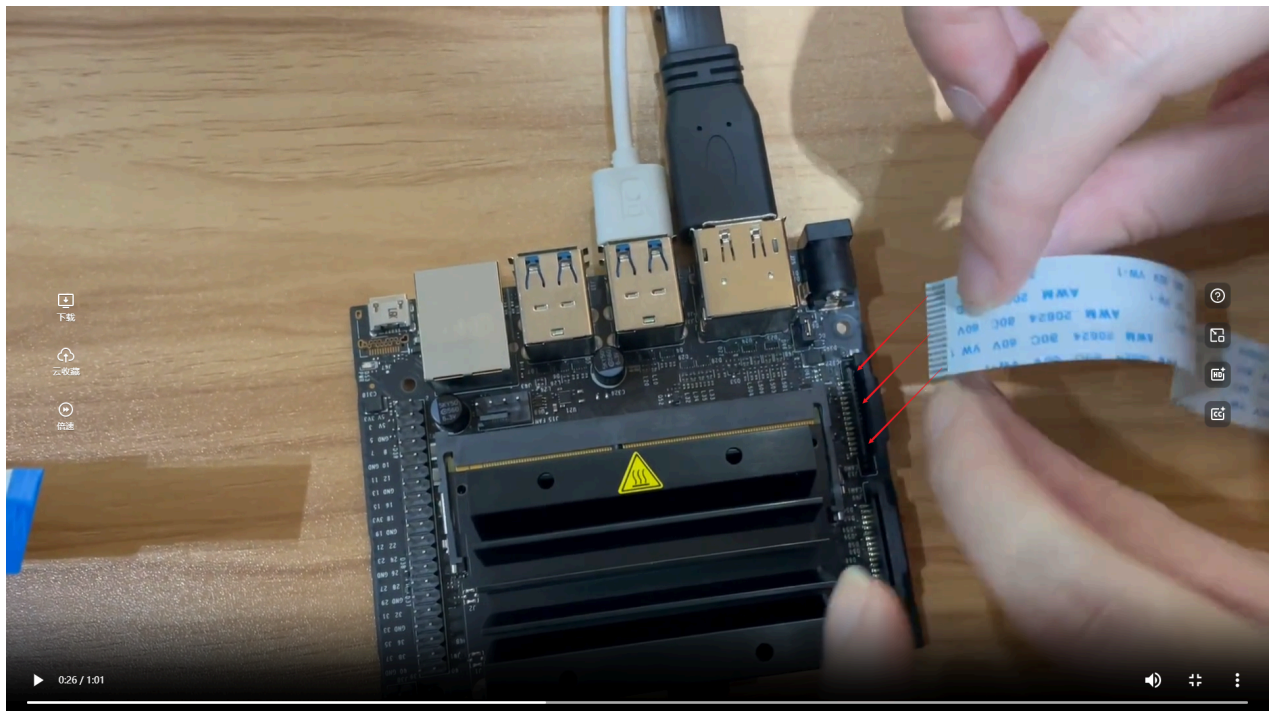
- OLED and temperature/humidity sensor are not external connections, they are onboard on the openclaw expansion board
- Servo wiring diagram, the S1 signal pin corresponds to yellow, red corresponds to 5V, brown corresponds to GND



- RGB light strip, anti-misinsertion interface, see diagram below



- CSI camera, align and insert into the slot.



3. DHT Hardware Test

Note: If you need to continuously read DHT through Python driver library, it is recommended to read DHT data at a 2-second interval for much better stability

3.1 Verification Method

```
dht    # Get sensor data
dht_oled # Display sensor data on OLED
```

Full command:

```
/usr/local/bin/dht --format json    # Get sensor data
/usr/local/bin/dht_oled --format json # Display sensor data on OLED
```

Normal input to terminal display, you can see OLED displaying temperature and humidity data

```

root@jetson-yahboom:~# dht
{"ok": true, "sensor": "dht11", "temperature_c": 29.9, "temperature_f": 85.8, "humidity": 52.0, "attempts": 1}
root@jetson-yahboom:~# dht_oled
{"ok": true, "code": "ok", "stage": "completed", "message": "温湿度已显示到 OLED", "reading": {"ok": true, "sensor": "dht11", "temperature_c": 29.9, "temperature_f": 85.8, "humidity": 52.0, "attempts": 1}, "oled": {"title": "DHT11", "text": "Temp:29.9C\\rHum :52.0%", "lines": ["DHT11", "Temp:29.9C", "Hum :52.0%"], "dry_run": false, "i2c_bus": "auto", "render_title": "DHT11", "render_text": "Temp:29.9C\\rHum :52.0%"}
root@jetson-yahboom:~#

```

4. RGB Hardware Test

4.1 Verification Method

Run in terminal:

```

rgb color --color red # All turn red
rgb effect --effect breathe --color blue --speed 5 # Blue breathing light

```

Full command:

```

/usr/local/bin/rgb --format json color --color red # All turn red
/usr/local/bin/rgb --format json effect --effect breathe --color blue --speed 5 # Blue breathing light

```

Normal input to terminal display, you can see RGB light displaying normally

```

root@jetson-yahboom:~# rgb color --color red
{"ok": true, "code": "ok", "message": "RGB 颜色已设置", "details": {"index": 255, "r": 255, "g": 0, "b": 0}}
root@jetson-yahboom:~# rgb effect --effect breathe --color blue --speed 5
{"ok": true, "code": "ok", "message": "RGB 特效已设置: breathe", "details": {"effect": "breathe", "speed": 5, "color": "blue"}}
root@jetson-yahboom:~#

```

Turn off light strip

```

rgb off

```

Full command:

```

/usr/local/bin/rgb --format json off

```

```

root@jetson-yahboom:~# rgb off
{"ok": true, "code": "ok", "message": "RGB 颜色已设置", "details": {"index": 255, "r": 0, "g": 0, "b": 0}}
root@jetson-yahboom:~#

```

5. Servo Hardware Test

5.1 Verification Method

Control single servo:

```

servo call-tool Servo1 --action set_angle --angle 90 # Servo1 rotates to 90 degrees
servo call-tool Servo2 --action set_angle --angle 180 # Servo2 rotates to 90 degrees

```

Full command:

```

/usr/local/bin/servo call-tool Servo1 --action set_angle --angle 90 # Servo1 rotates to 90
degrees
/usr/local/bin/servo call-tool Servo2 --action set_angle --angle 180 # Servo2 rotates to 90
degrees

# Servo1 is the servo to control
# 90 is the angle to rotate to

```

```

root@jetson-yahboom:~# servo call-tool Servo1 --action set_angle --angle 90
{"ok": true, "code": "ok", "message": "done", "device_id": "servo_1", "action": "set_angle", "request_id": "", "state": {"angle": 90.0, "attached": true, "last_angle": 90.0, "min_angle": 0.0, "max_angle": 180.0, "telemetry": {"mode": "direct", "driver": "openclaw_servo", "servo_id": 1, "min_angle": 0.0, "max_angle": 180.0, "action": "set_angle", "params": {"angle": 90.0}}, "retryable": false}}
root@jetson-yahboom:~# servo call-tool Servo2 --action set_angle --angle 180
{"ok": true, "code": "ok", "message": "done", "device_id": "servo_2", "action": "set_angle", "request_id": "", "state": {"angle": 180.0, "attached": true, "last_angle": 180.0, "min_angle": 0.0, "max_angle": 180.0, "telemetry": {"mode": "direct", "driver": "openclaw_servo", "servo_id": 2, "min_angle": 0.0, "max_angle": 180.0, "action": "set_angle", "params": {"angle": 180.0}}, "retryable": false}}
root@jetson-yahboom:~#

```

Synchronously control all servos:

```

# Servos simultaneously reach specified angles
servo call-tool ServoAll --action set_all --angle1 0 --angle2 60 --angle3 120 --angle4 180
# All servos return to center position
servo call-tool ServoAll --action set_all --angle1 90 --angle2 90 --angle3 90 --angle4 90

```

Note: ServoALL requires writing all servo angles. The following command is invalid:

```
servo call-tool ServoAll --action set_all --angle1 90 --angle2 90
```

If there are only two servos, the other two angles can be any value.

6. OpenClaw Firmware Serial Port Settings

6.1 Current Default Values

Current OpenClaw firmware serial port default parameters are:

- `port="/dev/ttyTHS1"`
- `baudrate=115200`

Other mainboard pin serial ports may not be named this way. For example, Raspberry Pi uses `"/dev/ttyAMA0"`

6.2 Modification Example

Edit configuration file:

```
nano /root/openclaw_port/hardware_hal_demo/hardware_config.py
```

Change to for Jetson nano:

```

"port": "/dev/ttyTHS1",
"baudrate": 115200,

```

```

1  #!/usr/bin/env python3
2  """硬件配置 - 简化版本, 只保留仍实际生效的默认值"""
3  from __future__ import annotations
4
5
6  class ConfigError(RuntimeError):
7      pass
8
9
10 # RGB 灯带默认配置
11 RGB_DEFAULTS = {
12     "pixel_count": 14,
13 }
14
15 # OpenClaw 固件默认配置
16 OPENCLAW_DEFAULTS = {
17     "port": "/dev/ttyTHS1",
18     "baudrate": 115200,
19 }
20
21 # OLED 显示屏默认配置
22 OLED_DEFAULTS = {
23     "enabled": True,
24     "width": 128,
25     "height": 32,
26     "interface": "i2c",
27     "i2c_bus": "auto",
28     "i2c_bus_candidates": [1, 0, 7, 8],
29     "i2c_address": 0x3C,
30     "rotate": 0,
31     "invert": False,
32     "contrast": 255,
33     "refresh_interval": 1.0,
34     "font": {
35         "path": "",
36         "size": 9,
37         "index": 0,
38         "fallback_path": "",
39     },
40     "layout": {
41         "padding_left": 0,

```

6.3 Verification Method

After saving, it is recommended to first test with either servo or RGB underlying command, for example:

```
rgb color --color green
```

Or:

```

servo call-tool Servo1 --action center # Servo1 returns to center 90 degrees
# All servos return to center position
servo call-tool ServoAll --action set_all --angle1 90 --angle2 90

```

If the command returns normally and can control peripherals, the serial port link is usable.

```

root@jetson-yahboom:~# servo call-tool ServoAll --action set_all --angle1 90 --angle2 90 --angle3 90 --angle4 90
{"ok": true, "code": "ok", "message": "done", "device_id": "all", "action": "set_all", "request_id": "", "state": {"angles": [90, 90, 90, 90], "telemetry": {"mode": "direct", "params": {"angle1": 90.0, "angle2": 90.0, "angle3": 90.0, "angle4": 90.0}}, "retryable": false}
root@jetson-yahboom:~#

```

7. OLED Hardware Test

7.1 Current Default Values

Current OLED default parameters mainly include:

- `width=128`
- `height=32`
- `interface="i2c"`
- `i2c_bus="auto"`
- `i2c_bus_candidates=[1, 0, 7, 8]`
- `i2c_address=0x3C`
- `rotate=0`
- `invert=False`
- `contrast=255`

7.2 Scenarios Suitable for Modification

It is recommended to modify OLED default values in these situations:

- You are using an OLED with different resolution
- I2C address is not `0x3C`
- Screen orientation needs to be flipped

7.3 Common Modification Items

Modify Resolution

For example, change:

```
"width": 128,  
"height": 32,
```

To:

```
"width": 128,  
"height": 64,
```

Modify I2C Address

For example, change:

```
"i2c_address": 0x3C,
```

To:

```
"i2c_address": 0x3D,
```

Modify Rotation Direction

For example, change:

```
"rotate": 0,
```

To:

```
"rotate": 180,
```

7.4 Verification Method

After saving, run in terminal:

```
oled text --text 'Hello OpenClaw'  
oled clear
```

Full command:

```
/usr/local/bin/oled text --text 'Hello OpenClaw'  
/usr/local/bin/oled clear
```

If OLED can display normally and clear the screen, the configuration has taken effect.

```
● root@jetson-yahboom:~# oled text --text 'Hello OpenClaw'  
  {"ok": true, "code": "ok", "mode": "text", "message": "文字已显示到 OLED", "details": {"title": "", "text": "Hello OpenClaw", "lines": ["Hello OpenClaw"], "dry_run": false}}  
● root@jetson-yahboom:~# oled clear  
  {"ok": true, "code": "ok", "mode": "clear", "message": "OLED 已清屏", "details": {"dry_run": false}}  
○ root@jetson-yahboom:~# █
```

8. Camera Hardware Test

8.1 Configuration

Prerequisite is to ensure CSI camera is connected. The current CSI camera chain mainly works through these two entry points:

```
/root/openclaw_port/hardware_hal_demo/camera  
/root/.openclaw/workspace/skills/csi-camera/scripts/csi_snap
```

8.2 Check Port

Input in terminal,

```
ls /dev/video*
```

Under normal circumstances, you can see video devices,


```
root@jetson-yahboom:~# ls /dev/video*  
/dev/video0  
root@jetson-yahboom:~#
```

8.3 Photo Verification Method

Input command in terminal, first directly test the photo chain:

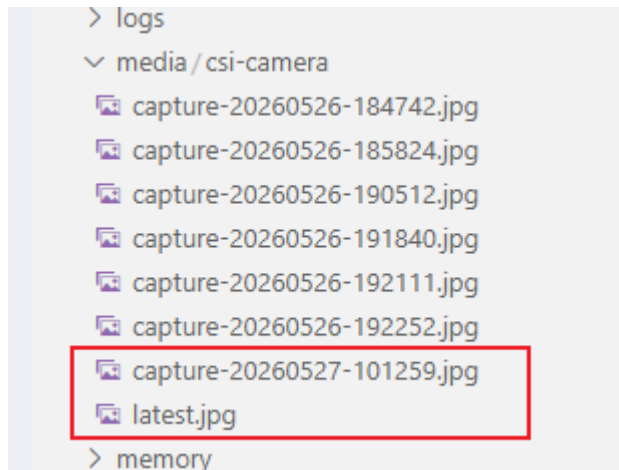
```
csi_snap snap
```

```
root@jetson-yahboom:~# csi_snap snap  
{\"ok\":true,\"code\":\"ok\",\"mode\":\"snap\",\"backend\":\"jetson-gstreamer\",\"camera_cmd\":\"gst-launch-1.0 nvarguscamerasrc\",\"saved_path\":\"/root/.openclaw/media/csi-camera/capture-20260526-145459.jpg\",\"latest_path\":\"/root/.openclaw/media/csi-camera/latest.jpg\",\"width\":1280,\"height\":720,\"timeout_ms\":1000,\"rotation\":0,\"timestamp\":\"20260526-145459\",\"capture_id\":\"cap-20260526-145459-790bcbaf9dc\",\"sha256\":\"790bcbaf9dc40462881a7b14e443662f80b0bcd46db1871656a2bb501a3699c\",\"sha256_12\":\"790bcbaf9dc\",\"size_bytes\":70965}  
root@jetson-yahboom:~#
```

Note: If the program gets stuck and the camera light stays on, this means the camera is not properly inserted. Power off and reinsert properly.

If photo is successful, you will usually see output files in the following directory:

```
/root/.openclaw/media/csi-camera
```



8.4 Adjustable Parameter Description

Although camera is currently not configured in `hardware_config.py`, `csi_snap` supports some photo parameters when called, such as:

- `--width`
- `--height`
- `--timeout-ms`
- `--rotation`
- `--hflip`
- `--vflip`
- `--camera`

For example:

```
csi_snap snap --width 1280 --height 720
```

```
root@jetson-yahboom:~# csi_snap snap --width 1280 --height 720
{"ok":true,"code":"ok","mode":"snap","backend":"jetson-gstreamer","camera_cmd":"gst-launch-1.0 nvarguscamerasrc","saved_path":"/root/.openclaw/media/csi-camera/capture-20260526-145613.jpg","latest_path":"/root/.openclaw/media/csi-camera/latest.jpg","width":1280,"height":720,"timeout_ms":1000,"rotation":0,"timestamp":"20260526-145613","capture_id":"cap-20260526-145613-3dca0a6ab886","sha256":"3dca0a6ab886656fc64195eef48cf686aa95db1be2cecf3963fc360620a11","sha256_12":"3dca0a6ab886","size_bytes":70961}
root@jetson-yahboom:~#
```

9. Development Environment Description:

Since the Jetson Nano B01 system version is 18.04, openclaw related features are all experienced in the docker container. To view folders in the docker container, this tutorial uses VScode v1_85 with the Release version Dev Containers and Remote - SSH plugin to enter the Jetson Nano B01's running docker container to view. The specific operations are as follows:

9.1 Download VScode

You can go to the VScode official website to download the corresponding version:

```
https://code.visualstudio.com/updates/v1\_85
```

9.2 Install Corresponding Plugins, Note as Pre-Release Version



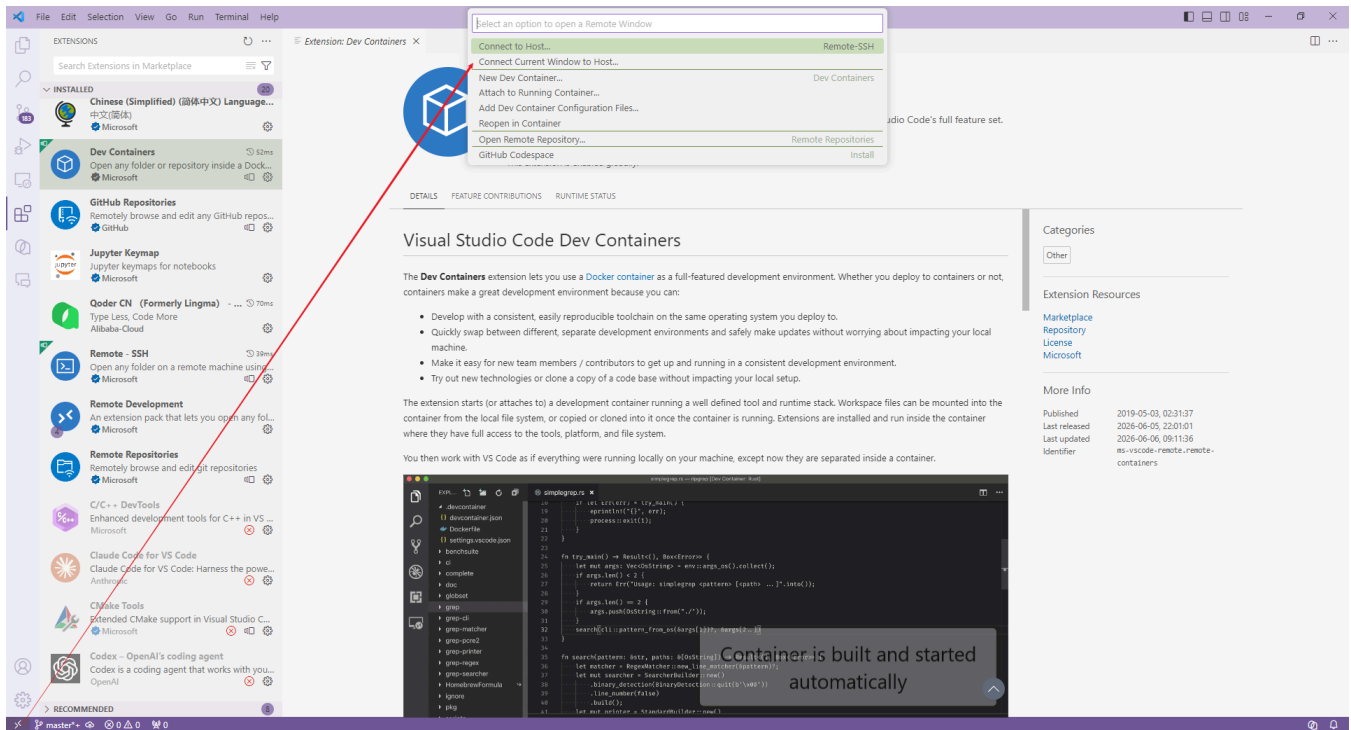
Remote - SSH v0.108.2023112915 Pre-Release
Microsoft  microsoft.com | 35,183,586 | ★★★★★ (213)
Open any folder on a remote machine using SSH and take advantage of VS Code's full feature set.
Disable Uninstall Switch to Release Version 
This extension is enabled globally.



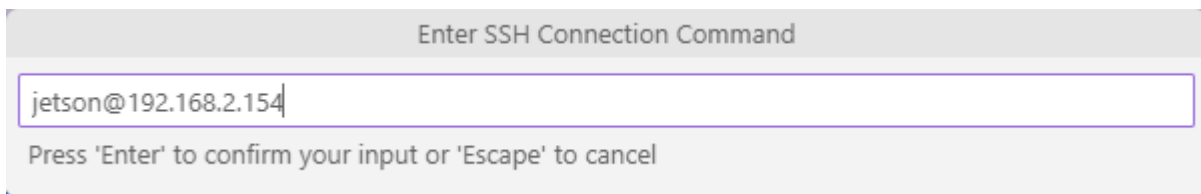
Dev Containers v0.330.0 Pre-Release
Microsoft  microsoft.com | 39,739,824 | ★★★★★ (58)
Open any folder or repository inside a Docker container and take advantage of Visual Studio Code's full feature set.
Disable Uninstall Switch to Release Version 
This extension is enabled globally.

9.3 Enter Docker Container

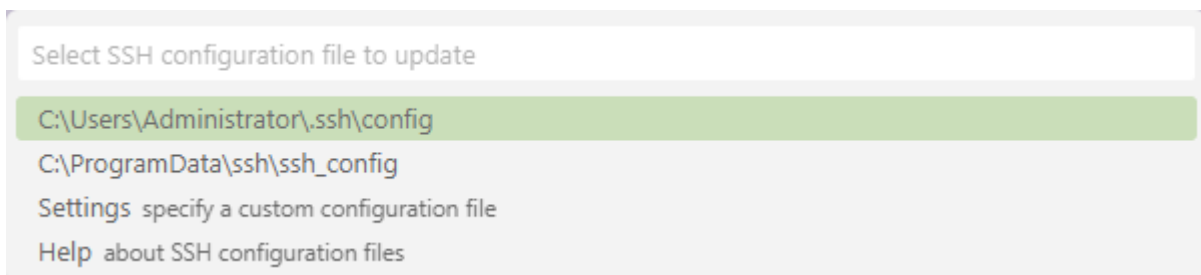
Click connect to remote, then select Connect to Host...

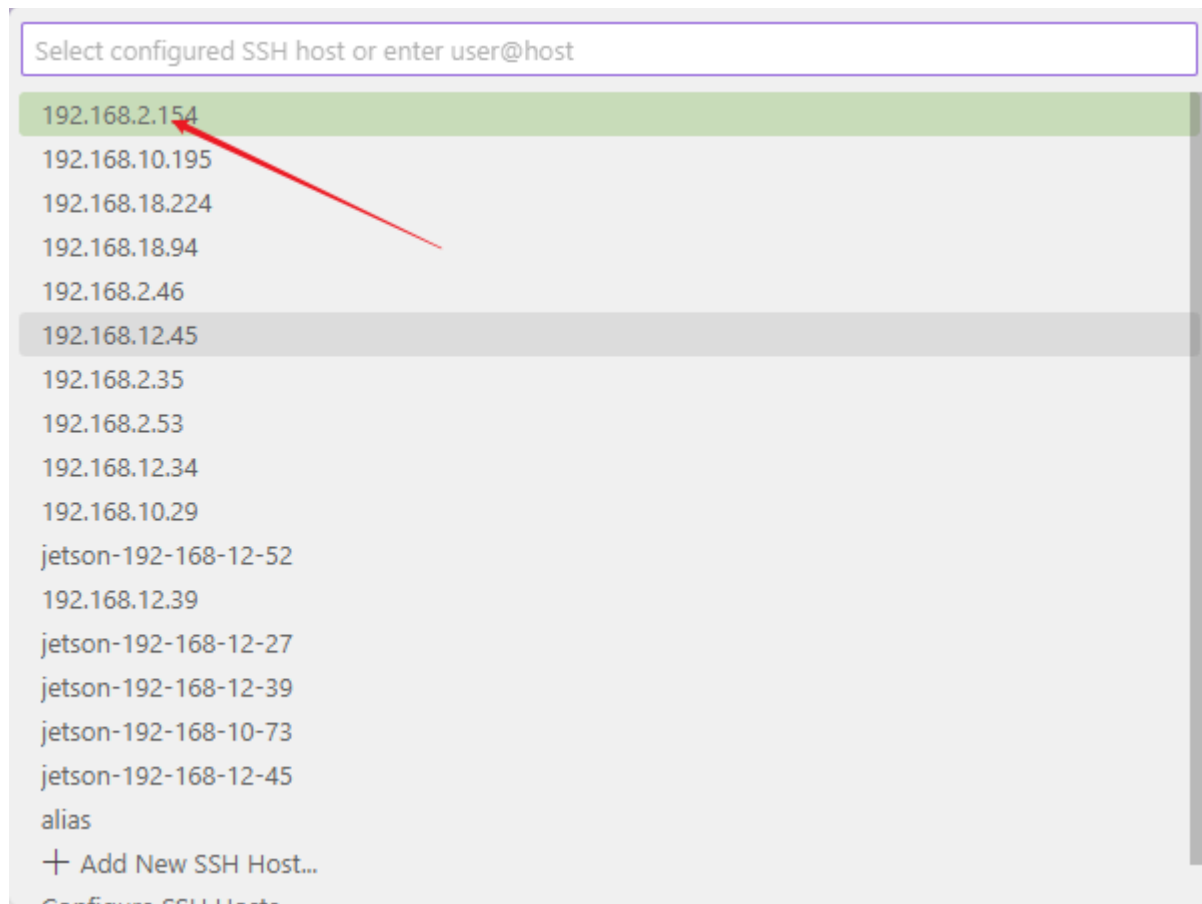


Select to add new SSH, input jetson@

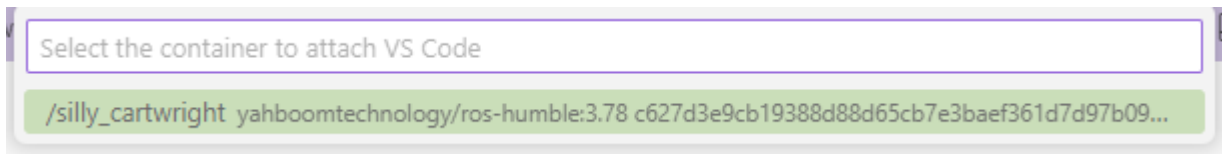
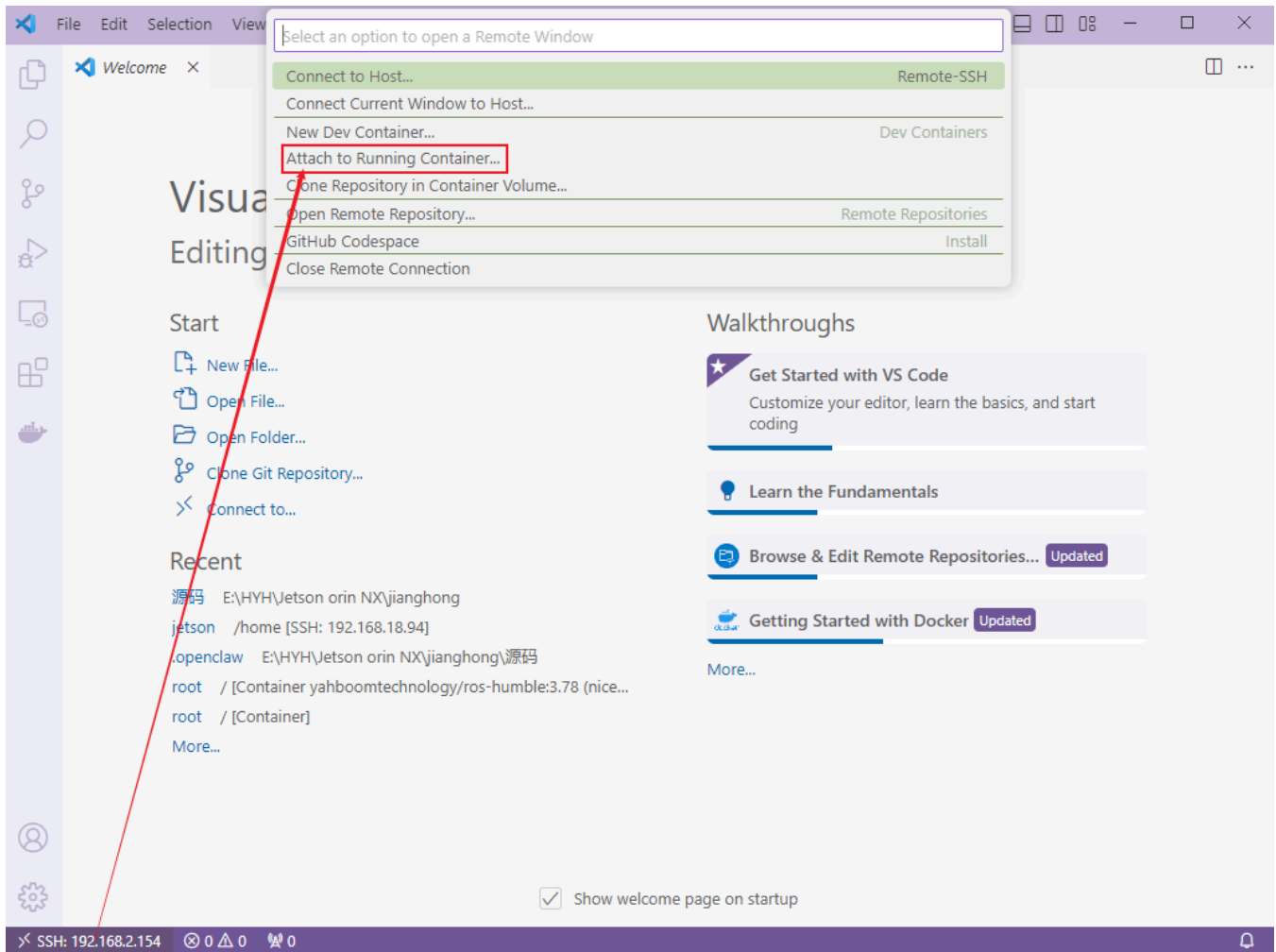


After pressing Enter, select the updated file location, then click connect to remote again. At this time you can see the device. Go inside, follow the instructions to select system and input password yahboom to enter the jetson nano system.





Then click connect to remote in the new terminal, this time select Attach to Running Container..., Note: The container must be started first before VScode can find the currently running container name!!!



After selecting to enter, follow the prompts to continue entering password yahboom. At this point you can enter the container's system to view files

